作业七说明

Homework

- Modify our RWLock implementation to use only one condition variable

- Implement Banker's Algorithm
  - Input-1: task number N, resource type number M;
  - Input-2: resource amount: for each type: R_i where i=1-M
  - Input-3: MAX resource for each task $<T_{i,j}>$ where i=1-N and j=1-M;
  - Input-4: Sequence of resource request $<R_{i,j}>$ where i=1-N and j=1-M
    - ❑ You can define your own way to generate this sequence

  - Test your algorithm with a large number of random sequences of resource request. Make sure deadlock never happens!

1. 把徐老师在课上讲的 RWLock 实现改成只用一个条件变量的实现。使用两个条件变量的实现在课件 pdf 的 53/54 页能看到。实现 RWLock 之后，还需要编写对这个锁的测试例程，比如可以是读写者模型的多线程程序。需要在提交的 pdf 里面描述自己的测试例程为什么能证明 RWLock 的功能是正常的。
   实现语言不做限制。
   Bonus：
   - （easy）用多种语言实现，比如 c 和 cpp 或者 rust？实现之后依然需要编写测试例程，并在文档中说明为什么测试可以证明 RWLock 是正常的。
   - （medium）尝试使用 CAS（Compare and swap）无锁实现 RWLock，实现语言不限。
   - （hard）深入了解原子操作、内存屏障（包括编译器级别和 CPU 级别），然后理解内核里面的 RCU（Read Copy Update）机制，用文字详细说明自己的理解。**要求：文字描述严谨、准确，如果能结合源码分析是最好的。提交粗浅的描述不算完成该 bonus。**

   提交内容：代码+文档

2. 实现银行家算法，编写随机生成测试序列的脚本，需要有至少两个随机序列：
   a) 完全随机生成的申请序列。序列只满足基本的要求，比如申请的资源比任务当前需要的最大资源数少、申请的任务编号属于 1-N、等等
   b) 可以让所有任务正常完成的申请序列。执行该序列的过程中，或者序列执行完毕时，所有任务已经正常完成。

   序列长度不限制，但是至少是十数量级。任务数量和资源类型、数量可以是十以内的。

需要在文档里面描述自己实现的算法细节，怎么执行测试程序，随机序列是怎么生成的，执行测试程序的预期结果及解释。

提交内容：代码+文档